

Project 1: 16-Bit Processor

Checkpoint 1 Report

Jacob Bemis — jacobbem@buffalo.edu

Allie Victorino — allievic@buffalo.edu

Project Github: https://github.com/dustbunnz/cse490_project1/

Logbook of Meeting and Discussions

Date	Work Done	Meeting Information
9/20	• Planning and work division • Weekly log document created • Work division doc created	We divided the work and discussed what parts each of us should focus on
10/4	Jacob: • Created source files and port map for Program Counter, Instruction Memory, Data Memory, and Control Unit. • Started working on the behavioral design for Program Counter and Instruction Memory Allie: • Started source file for Register File. • Started top level design for Register File and ALU	No meeting today

Distribution of Work

Allie:

- Register File
- ALU
- Sign Extension
- Multiplexers
- R type instructions and addi
- Corresponding Testbenches
- Corresponding Datapaths

Jacob:

- Program Counter

- Instruction Memory
- Data Memory
- Control Unit
- Branch/Jump instructions
- Corresponding Testbenches
- Corresponding datapaths

Components Worked On

Program Counter:

The Verilog file for the Program Counter was created named “program_counter.v”. In this file, the port map has defined the inputs and outputs for the Program Counter. The inputs consist of a clock signal, reset, and the address of the next PC. The logic for calculating the next PC value will be built later. For now, the logic goes as follows: If the reset signal is 1, then the output will be the address, 0x0000, else, the output will be the next PC.

Instruction Memory:

The Verilog file for the Instruction Memory was created named “instr_mem.v”. The port map was created, where the input is a 16-bit address, and the output is a 16-bit value representing the instruction. So far, the code outlines the future plans for the logic of this block. Using \$readmemh, a memory array will be loaded from a text file, containing the instructions. The output is set to the instruction located at the address in the memory array.

Data Memory:

The Verilog file for the Data Memory was created named “data_mem.v”. All that has been done for this component is the port map. The inputs are a 16-bit address, the write data, and the signals for MemWrite and MemRead. The output is Read Data.

Control Unit:

The Verilog file for the Control Unit was created named “control_unit.v”. All that has been done for this component is the port map. The inputs are the opcode and the function fields from an instruction. The outputs are all the control signals: RegDst, Branch, MemRead, MemToReg, ALUOP, MemWrite, ALUSrc, and RegWrite.

Register File:

The Verilog file for the Register File was created and named “registerfile.v”. The port map and the basic structure are completed, as well as a general idea of what will be implemented after the top level design is completed. The inputs are the clock signal, the regwrite flag, readreg_1, readreg_2, write_reg, and write_data. The outputs are read_data1 and read_data2. For the outputs/inputs, the MIPS register file datapath and the example datapath given in the pointers were referenced, however the final datapath will be refactored and fleshed out after completing the top level design and some work on the ALU.